



UANL

UNIVERSIDAD AUTÓNOMA DE NUEVO LEÓN



FIME

FACULTAD DE INGENIERÍA MECÁNICA Y ELÉCTRICA

UNIVERSIDAD AUTÓNOMA DE NUEVO LEÓN
FACULTAD DE INGENIERÍA MECÁNICA Y ELÉCTRICA
LENGUAJES DE PROGRAMACIÓN Y LAB
DOCENTE: ING. SELENE GUADALUPE PINAL TORRES
ACTIVIDAD FUNDAMENTAL 6
EQUIPO: 04

Matricula	Nombre	Carrera
2103677	Ruedas Vazquez Jordan Emanuel	ITS
2042781	Enríquez Nungaray Héctor Miguel	IAS
1975208	Hernández Silva Carlos Yahir	ITS
2054413	Heredia López Betsy Aracely	IAS

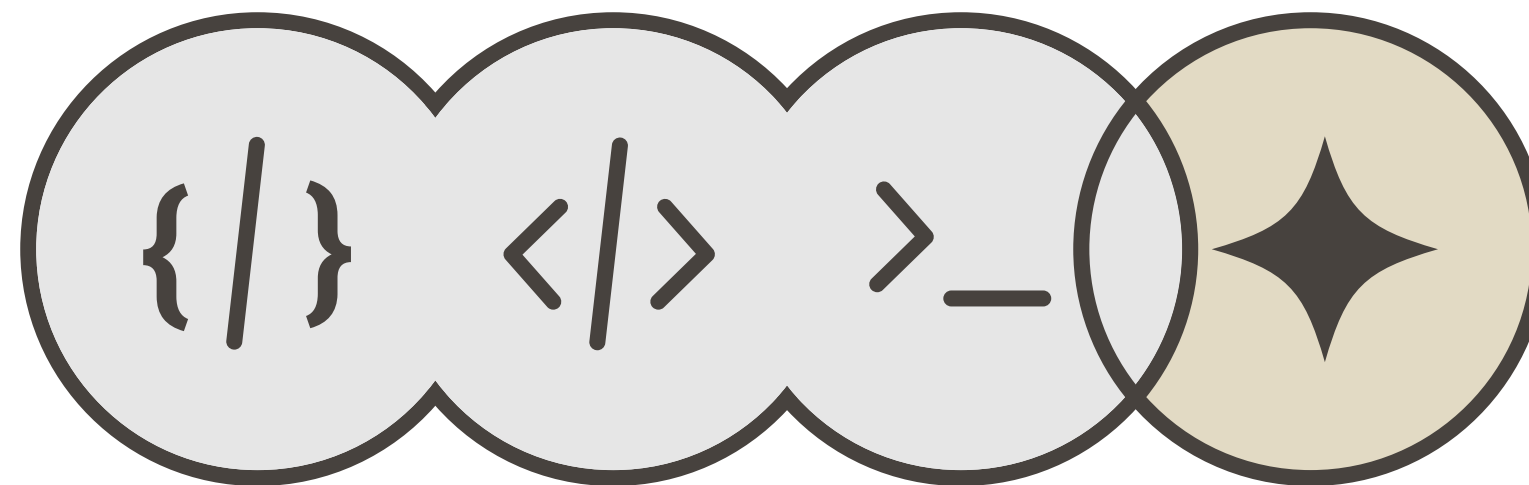
MONTERREY, NUEVO LEÓN

2 — LENGUAJE FUNCIONAL

ACTIVIDAD FUNDAMENTAL 6

INTRODUCCIÓN

La programación funcional es un paradigma de programación basado en el uso de funciones como elementos fundamentales para construir software. A diferencia de otros enfoques, como la programación imperativa u orientada a objetos, la programación funcional se centra en la evaluación de funciones puras, la inmutabilidad de los datos y la ausencia de efectos secundarios. Este enfoque permite escribir código más predecible, fácil de probar y mantener, lo que lo convierte en una opción atractiva para resolver problemas complejos, especialmente en entornos concurrentes y distribuidos.



CARACTERÍSTICAS PRINCIPALES



- Funciones puras

Las funciones puras siempre producen el mismo resultado cuando se les da el mismo conjunto de entradas y no generan efectos secundarios. Esto significa que no modifican variables externas ni interactúan con el estado del programa fuera de su ámbito.

- Inmutabilidad

En programación funcional, los datos son inmutables: una vez creados, no pueden cambiar. En lugar de modificar estructuras de datos existentes, se crean nuevas versiones con los cambios necesarios, lo que mejora la seguridad y previsibilidad del código.

- Funciones como ciudadanos de primera clase

Las funciones se tratan como valores: pueden pasarse como argumentos, retornarse desde otras funciones y almacenarse en estructuras de datos. Esto permite construir programas más flexibles y reutilizables.

- Transparencia referencial

Un fragmento de código se puede reemplazar por su valor sin cambiar el comportamiento del programa. Esta propiedad facilita el razonamiento, la prueba y la optimización del código.

- Evaluación perezosa (lazy evaluation)

En algunos lenguajes funcionales, las expresiones no se evalúan hasta que su resultado es necesario. Esto permite trabajar con estructuras de datos infinitas y optimizar el rendimiento.

DIFERENCIAS CON OTROS PARADIGMAS

Estado vs. Inmutabilidad

- POO/Imperativa: Se basa en el manejo de estados mutables, donde los objetos pueden cambiar su información interna a lo largo del tiempo.
- Funcional: Promueve la inmutabilidad, evitando cambios de estado. Los datos no se modifican, sino que se generan nuevas versiones cuando es necesario.

Funciones vs. Objetos

- POO: El foco está en los objetos, que combinan datos y comportamiento (métodos).
- Funcional: El centro son las funciones puras, que transforman datos sin depender de un estado externo.



Efectos secundarios

- POO/Imperativa: Es común que las funciones o métodos tengan efectos secundarios (como modificar una variable global o escribir en un archivo).
- Funcional: Se evitan los efectos secundarios, lo que hace que el comportamiento del programa sea más predecible.

DEFINICIÓN DE LENGUAJE FUNCIONAL

¿Qué es el lenguaje funcional?

Es un paradigma de programación basado en el uso de funciones como el modelo central para construir software. Se inspira en las funciones matemáticas, donde cada conjunto de entradas produce una única salida, sin depender de estados externos. A diferencia de la programación imperativa, se enfoca en qué se quiere lograr, no en cómo hacerlo. Evita el uso de datos mutables, efectos secundarios y estados compartidos, permitiendo así mayor claridad, predictibilidad y facilidad para probar el código.



DEFINICIÓN DE LENGUAJE FUNCIONAL



Principios clave:

- **Cálculo lambda:** Todo se representa usando funciones matemáticas.
- **Declarativo:** Se dice qué hacer, no cómo hacerlo paso a paso.
- **Funciones puras:** Siempre dan el mismo resultado sin cambiar nada fuera.
- **Funciones de primera clase:** Se pueden usar como valores, pasarlas o devolverlas.
- **Inmutabilidad:** Las variables no cambian una vez creadas.
- **Sin efectos secundarios:** No afecta nada fuera de la función.
- **Recursión:** Usa funciones que se llaman a sí mismas en lugar de bucles.
- **Composición:** Las funciones se pueden unir para formar otras más complejas.

LENGUAJES FUNCIONALES

Inmutabilidad:

Las variables no pueden ser modificadas una vez que se han inicializado. Si se requiere almacenar un nuevo valor, se define una nueva variable. Esta propiedad garantiza que el estado del programa no cambiará inesperadamente, lo cual mejora la confiabilidad y facilita el razonamiento del código.

Funciones puras:

Son funciones que, al recibir los mismos parámetros, siempre devuelven el mismo resultado y no alteran el estado del sistema ni tienen efectos secundarios, como modificar variables externas o interactuar con el sistema de archivos. Esto las hace más fáciles de probar, componer y mantener.

LENGUAJES FUNCIONALES

Recursión:

En lugar de usar bucles como `for` o `while`, la programación funcional utiliza funciones recursivas que se llaman a sí mismas hasta alcanzar un caso base. La recursión es el mecanismo natural para iterar en este paradigma.

Funciones de orden superior:

Son funciones que pueden recibir otras funciones como parámetros o devolver funciones como resultado. Esto permite construir operaciones complejas de forma más expresiva y concisa. Por ejemplo, funciones como `filter`, `map` o `reduce` en muchos lenguajes funcionales permiten trabajar con colecciones de datos sin necesidad de ciclos explícitos.

LENGUAJES FUNCIONALES POPULARES

Haskell

Ventajas

- Código muy limpio y conciso
- Excelente para programación pura
- Evaluación perezosa

DESVENTAJAS

- Curva de aprendizaje alta
- Comunidad más pequeña comparada con otros lenguajes

Erlang

Ventajas

- Ideal para sistemas concurrentes y distribuidos
- Muy tolerante a fallos

DESVENTAJAS

- Sintaxis poco intuitiva
- No es el más rápido en tareas computacionales intensivas



Clojure

Ventajas -

- Funciona sobre la JVM
- Enfocado en la inmutabilidad y concurrencia
- Sintaxis simple basada en Lisp

Desventajas

- Sintaxis de tipo "parenthesis everywhere" puede ser confusa
- Menor popularidad que Java

LENGUAJES FUNCIONALES POPULARES

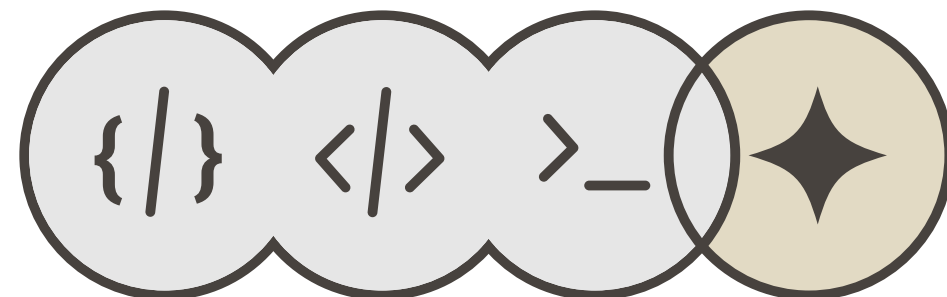
Scala

Ventajas

- Interoperable con Java
- Soporta programación funcional y orientada a objetos

DESVENTAJAS

- Puede ser complejo por su flexibilidad
- Compilación lenta



F#

VENTAJAS

- Integración con .NET
- Sintaxis clara y expresiva
- Buen soporte en Visual Studio

Desventajas

- Menos adoptado fuera del ecosistema Microsoft
- Documentación menos abundante comparado con otros lenguajes

APLICACIONES PRÁCTICAS



🌐 Procesamiento de Datos

- Big Data: Hadoop (MapReduce) y Apache Spark (Scala) para análisis distribuido.
- Ejemplos: Clojure (manejo de flujos), F# (transformaciones complejas).

🤖 Inteligencia Artificial

- Machine Learning: Scala (modelos estadísticos), Haskell (investigación en IA).
- Histórico: Lisp en sistemas expertos y procesamiento simbólico.

⚡ Sistemas Concurrentes

- Alta concurrencia: Erlang en WhatsApp/RabbitMQ (millones de conexiones).
- Tolerancia a fallos: Modelo de actores y sistemas distribuidos.

💰 Finanzas y Seguridad

- Trading algorítmico: OCaml (precisión sin errores de estado).
- Verificación formal: Agda/Idris para demostrar corrección de código (aeroespacial).

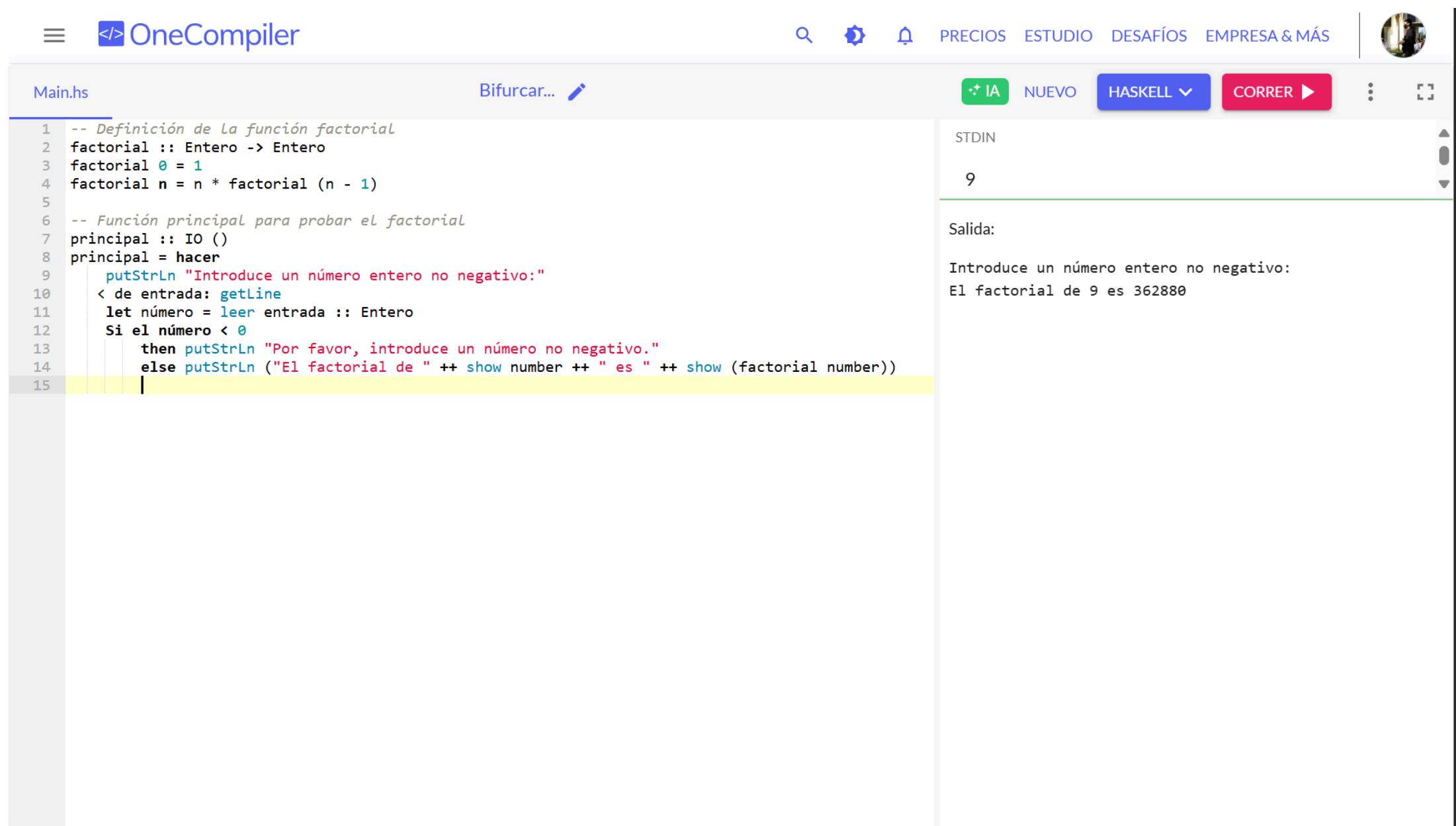
💻 Frontend Moderno

- Interfaces predecibles: React (JavaScript funcional), Elm (aplicaciones web robustas).



EJEMPLO

RUEDAS VAZQUEZ JORDAN EMANUEL



The screenshot shows the OneCompiler web interface. On the left, a Haskell code editor displays a recursive factorial function. The code defines a function `factorial` that takes an integer and returns an integer, with a base case of 1 for 0 and a recursive case `n * factorial (n - 1)`. A `principal` function handles user input, prompts for a non-negative number, and prints the result. The right side of the interface shows the execution environment with 'STDIN' input of '9' and 'Salida' (output) showing the prompt and the result 'El factorial de 9 es 362880'.

```
1 -- Definición de la función factorial
2 factorial :: Entero -> Entero
3 factorial 0 = 1
4 factorial n = n * factorial (n - 1)
5
6 -- Función principal para probar el factorial
7 principal :: IO ()
8 principal = hacer
9   putStrLn "Introduce un número entero no negativo:"
10 < de entrada: getLine
11 let número = leer entrada :: Entero
12 Si el número < 0
13   then putStrLn "Por favor, introduce un número no negativo."
14   else putStrLn ("El factorial de " ++ show número ++ " es " ++ show (factorial número))
15
```

STDIN

9

Salida:

Introduce un número entero no negativo:
El factorial de 9 es 362880

Variables y Flujo

- 1.number (Variable global):
 - Tipo: Integer.
 - Función: Almacena el valor ingresado por el usuario después de convertir input (texto) a número.
- 2.n (Parámetro de la función factorial):
 - Tipo: Integer.
 - Función: Representa el número del que se calcula el factorial. Se reduce en cada llamada recursiva (n-1) hasta llegar al caso base (n=0).

Flujo de Datos

- 1.Entrada:
 - El usuario introduce un número (ej: 5).
 - input captura el texto, y number lo convierte a Integer.
- 2.Validación:
 - Si number < 0, muestra un mensaje de error.
 - Si number >= 0, calcula factorial number.
- 3.Cálculo Recursivo:
 - Ejemplo para number = 3:
 - factorial 3 = 3 * factorial 2
 - factorial 2 = 2 * factorial 1
 - factorial 1 = 1 * factorial 0
 - factorial 0 = 1
 - Resultado final: 3 * 2 * 1 * 1 = 6.
- 4.Salida:
 - Imprime el resultado (ej: "El factorial de 3 es 6").

EJEMPLO

ENRÍQUEZ NUNGARAY HÉCTOR MIGUEL

```
1 <!DOCTYPE html>
2 <html>
3 <head>
4   <title>Calculadora de Áreas</title>
5   <style>
6     body {
7       font-family: Arial;
8       background-color: #f2f2f2;
9       padding: 20px;
10    }
11    input, button {
12      padding: 10px;
13      margin: 5px 0;
14      font-size: 16px;
15    }
16    .resultado {
17      margin-top: 20px;
18      background-color: #fff;
19      padding: 15px;
20      border-radius: 8px;
21      box-shadow: 0 2px 6px rgba(0,0,0,0.1);
22    }
23  </style>
24 </head>
25 <body>
26
27   <h2>📐 Calculadora de Áreas</h2>
28
29   <label>Base:</label><br>
```

```
30   <input type="number" id="base"><br>
31
32   <label>Altura:</label><br>
33   <input type="number" id="altura"><br>
34
35   <label>Radio:</label><br>
36   <input type="number" id="radio"><br><br>
37
38   <button onclick="calcularAreas()">Calcular Áreas</button>
39
40   <div class="resultado" id="resultado"></div>
41
42   <script>
43     function calcularAreaRectangulo(base, altura) {
44       return base * altura;
45     }
46
47     function calcularAreaTriangulo(base, altura) {
48       return (base * altura) / 2;
49     }
50
51     function calcularAreaCirculo(radio) {
52       return Math.PI * Math.pow(radio, 2);
53     }
54
55     function calcularAreas() {
56       let base = parseFloat(document.getElementById("base").value);
```

EJEMPLO: HECTOR MIGUEL ENRIQUEZ NUNGARAY

```
57 let altura = parseFloat(document.getElementById("altura").value);
58 let radio = parseFloat(document.getElementById("radio").value);
59
60 if (isNaN(base) || isNaN(altura) || isNaN(radio)) {
61     document.getElementById("resultado").innerHTML = " ! Por favor, ingresa todos los valores.";
62     return;
63 }
64
65 let areaRect = calcularAreaRectangulo(base, altura);
66 let areaTri = calcularAreaTriangulo(base, altura);
67 let areaCirc = calcularAreaCirculo(radio);
68
69 document.getElementById("resultado").innerHTML = `
70     ♦ Área del Rectángulo: <b>${areaRect.toFixed(2)}</b><br>
71     ♦ Área del Triángulo: <b>${areaTri.toFixed(2)}</b><br>
72     ♦ Área del Círculo: <b>${areaCirc.toFixed(2)}</b>
73 `;
74 }
75 </script>
76
77 </body>
78 </html>
```

Explicación del Código

1. Este código es una calculadora de áreas hecha con HTML y JavaScript. El usuario escribe la base, altura y radio en tres cajitas, y al hacer clic en el botón:
2. Se leen los valores ingresados.
3. Se calculan:
4. El área de un rectángulo ($\text{base} \times \text{altura}$)
5. El área de un triángulo ($((\text{base} \times \text{altura}) \div 2)$)
6. El área de un círculo ($\pi \times \text{radio}^2$)
7. Se muestran los resultados en pantalla con dos decimales.

Calculadora de Áreas

Base:

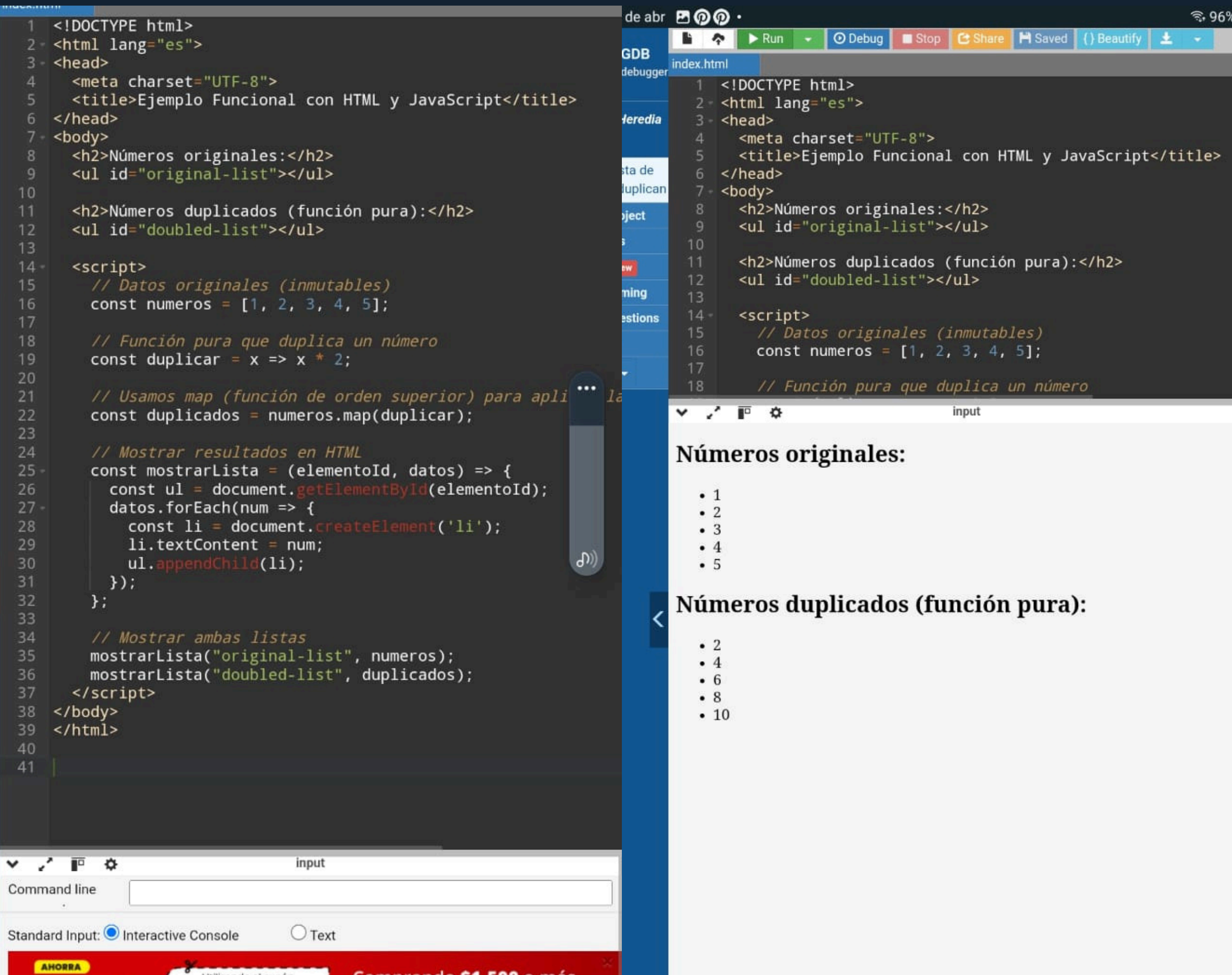
Altura:

Radio:

Calcular Áreas

- ♦ Área del Rectángulo: **300.00**
- ♦ Área del Triángulo: **150.00**
- ♦ Área del Círculo: **11309.73**

EJEMPLO: BETSY ARACELY HEREDIA LOPEZ



```
1 <!DOCTYPE html>
2 <html lang="es">
3 <head>
4   <meta charset="UTF-8">
5   <title>Ejemplo Funcional con HTML y JavaScript</title>
6 </head>
7 <body>
8   <h2>Números originales:</h2>
9   <ul id="original-list"></ul>
10
11  <h2>Números duplicados (función pura):</h2>
12  <ul id="doubled-list"></ul>
13
14  <script>
15    // Datos originales (inmutables)
16    const numeros = [1, 2, 3, 4, 5];
17
18    // Función pura que duplica un número
19    const duplicar = x => x * 2;
20
21    // Usamos map (función de orden superior) para aplicar la función a cada elemento
22    const duplicados = numeros.map(duplicar);
23
24    // Mostrar resultados en HTML
25    const mostrarLista = (elementoId, datos) => {
26      const ul = document.getElementById(elementoId);
27      datos.forEach(num => {
28        const li = document.createElement('li');
29        li.textContent = num;
30        ul.appendChild(li);
31      });
32    };
33
34    // Mostrar ambas listas
35    mostrarLista("original-list", numeros);
36    mostrarLista("doubled-list", duplicados);
37  </script>
38 </body>
39 </html>
40
41
```

Explicación del Código

- Se crean dos listas `<ul>` vacías en el HTML con identificadores (id) para poder llenarlas desde JavaScript.
- Una lista mostrará los números originales, y la otra los duplicados
- Se define un arreglo con números del 1 al 5.
- La variable `numeros` es constante (inmutable), lo cual es una buena práctica en programación funcional.
- Esta es una función pura: siempre que le pases el mismo valor `x`, te devolverá el mismo resultado (`x * 2`).
- No modifica ninguna variable externa ni produce efectos secundarios.
- Aquí usamos `map`, una función de orden superior que recorre el arreglo `numeros` y aplica la función `duplicar` a cada elemento.
- El resultado es un nuevo arreglo `duplicados` sin modificar el original.
- Esta función recibe un id de lista (`ul`) y un arreglo de datos.
- Usa `forEach` para recorrer los datos y crear elementos `<li>` para mostrarlos en la página.
- Aunque interactúa con el DOM (efecto secundario), la transformación de los datos ocurre de forma pura antes

EJEMPLO

HERNÁNDEZ SILVA CARLOS YAHIR

The screenshot shows the Spyder Python IDE interface. The main editor displays a Python script that uses functional programming concepts. The script defines a list of numbers, filters for even numbers, squares them, and then sums the results using the `reduce` function from `functools`. The output is printed to the console.

```
1 from functools import reduce
2
3 # Definir algunas funciones puras
4 square = lambda x: x * x
5 is_even = lambda x: x % 2 == 0
6 add = lambda x, y: x + y
7
8 # Lista de números para trabajar
9 numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
10
11 # Aplicar transformaciones funcionales encadenadas:
12 # 1. Filtrar solo números pares
13 # 2. Elevar al cuadrado cada número
14 # 3. Sumar todos los resultados
15 result = reduce(add, map(square, filter(is_even, numbers)))
16
17 print(f"Números originales: {numbers}")
18 print(f"Resultado final (suma de cuadrados de números pares): {result}")
19
```

The right sidebar shows the 'Explorador de variables' (Variable Explorer) with the following data:

Nombre	Tipo	Tamaño	Valor
numbers	list	10	[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
result	int	1	220

The bottom panel shows the 'Terminal 1/A' with the following output:

```
In [5]: %runfile 'C:/Users/carlo/OneDrive/Documentos/Sin título1.py' --wdir
Números originales: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
Resultado final (suma de cuadrados de números pares): 220

In [6]:
```

Este código usa programación funcional para trabajar con una lista de números.

Filtra los pares, eleva al cuadrado cada uno, y suma los resultados con `reduce`.

Todo con funciones puras (`lambda`). El resultado es la suma de los cuadrados de los números pares.

CONCLUSIÓN GENERAL

La programación funcional representa un enfoque poderoso y cada vez más relevante en el desarrollo de software, especialmente en contextos que requieren claridad, mantenibilidad y concurrencia. Al centrarse en funciones puras, la inmutabilidad y la ausencia de efectos secundarios, este paradigma permite escribir código más predecible, modular y fácil de probar. Aunque difiere notablemente de paradigmas como la programación orientada a objetos, ambos enfoques no son excluyentes y pueden complementarse según las necesidades del proyecto. Comprender las características, ventajas y limitaciones de la programación funcional permite a los desarrolladores tomar decisiones más informadas y mejorar la calidad de sus aplicaciones.

Los lenguajes de programación han evolucionado a lo largo de la historia para facilitar la interacción entre los programadores y las computadoras, permitiendo el desarrollo de software más eficiente y accesible. Desde los primeros lenguajes de bajo nivel como el ensamblador hasta las modernas tecnologías de alto nivel como Python y JavaScript, el objetivo ha sido siempre mejorar la productividad y la capacidad de resolución de problemas. Gracias a los avances en los paradigmas de programación, hoy en día existen lenguajes especializados para diversas áreas, desde la inteligencia artificial hasta el desarrollo web, cada uno optimizado para tareas específicas. Uno de los aspectos más importantes en la programación es la manera en que los lenguajes traducen las instrucciones humanas a código máquina. En este sentido, los compiladores e intérpretes juegan un papel crucial en la ejecución del software. Mientras que los compiladores generan programas altamente optimizados y rápidos, los intérpretes brindan mayor flexibilidad y facilidad de depuración.

CONCLUSIONES INDIVIDUALES

RUEDAS VAZQUEZ JORDAN EMANUEL

Los lenguajes de script y funcionales, aunque distintos en enfoque, comparten un objetivo común: simplificar la resolución de problemas mediante abstracciones eficientes. Los primeros, como Python o JavaScript, destacan por su agilidad y accesibilidad, democratizando la automatización de tareas cotidianas y el desarrollo rápido de aplicaciones web.

ENRÍQUEZ NUNGARAY HÉCTOR MIGUEL

Los lenguajes de scripting son eficientes para automatizar tareas y desarrollar aplicaciones rápidamente gracias a su simplicidad, mientras que los lenguajes funcionales se destacan por su enfoque en funciones puras e inmutabilidad, lo que favorece la seguridad, la legibilidad del código y el manejo del paralelismo. Ambos tipos de lenguajes tienen aplicaciones específicas según las necesidades del proyecto, ofreciendo ventajas únicas en distintos contextos de desarrollo.

HERNÁNDEZ SILVA CARLOS YAHIR

Los lenguajes de script han cobrado gran relevancia en el desarrollo de tecnologías por su rapidez, facilidad de uso y eficiencia. Su evolución continua los mantiene como herramientas clave en el desarrollo moderno.

Por otro lado, la programación funcional, común en muchos lenguajes de script, permite escribir código más legible, conciso y fácil de probar. Al enfocarse en funciones puras y evitar efectos secundarios, el desarrollo se vuelve más predecible y mantenible, facilitando tanto la depuración como el mantenimiento del software.

HEREDIA LÓPEZ BETSY ARACELY

Ambos tipos de lenguajes cumplen roles importantes en distintos contextos. Mientras que los lenguajes de scripting priorizan la agilidad y la facilidad de uso, los lenguajes funcionales destacan por su solidez teórica y capacidad de manejar la complejidad mediante principios como la inmutabilidad y las funciones puras.

Comprender las ventajas de cada enfoque permite a los desarrolladores elegir la herramienta adecuada para cada problema, combinar paradigmas de manera efectiva y diseñar soluciones más eficientes y sostenibles en el tiempo.

REFERENCIAS BIBLIOGRAFICAS

- CAIRO, O. (2024). ALGORITMOS Y ESTRUCTURAS DE DATOS. EDITORIAL UNIVERSITARIA.
- MONUS, A. (2015, 14 DE FEBRERO). LOS 13 PRINCIPALES LENGUAJES DE SCRIPTS A LOS QUE DEBERÍAS PRESTAR ATENCIÓN EN 2025. KINSTA®. [HTTPS://KINSTA.COM /ES/BLOG/LENGUAJES-SCRIPT/](https://kinsta.com/es/blog/linguajes-script/)
- MATTHES, E. (2019). PYTHON CRASH COURSE: A HANDS-ON, PROJECT-BASED INTRODUCTION TO PROGRAMMING (2.A ED.). NO STARCH PRESS.
- CROCKFORD, D. (2008). JAVASCRIPT: THE GOOD PARTS. O'REILLY MEDIA.
- PHP GROUP. (2023). PHP: HYPERTEXT PREPROCESSOR. [HTTPS://WWW.PHP.NET /MANUAL/ES/INTRO-WHATIS.PHP](https://www.php.net/manual/es/intro-what-is.php)
- IEEE COMPUTER SOCIETY. (2021). TRENDS IN SCRIPTING LANGUAGES: FLEXIBILITY VS. PERFORMANCE. IEEE XPLORE. [HTTPS://IEEEEXPLORE.IEEE.ORG /DOCUMENT/9472331](https://ieeexplore.ieee.org/document/9472331)
- MOZILLA DEVELOPER NETWORK. (2024). JAVASCRIPT GUIDE. [HTTPS://DEVELOPER.MOZILLA.ORG/ES/DOCS/WEB/JAVASCRIPT/GUIDE](https://developer.mozilla.org/es/docs/web/javascript/guide)
- TYPESCRIPT HANDBOOK. (2024). BASIC TYPES. [HTTPS://WWW.TYPESCRIPTLANG.ORG/DOCS/HANDBOOK/BASIC-TYPES.HTML](https://www.typescriptlang.org/docs/handbook/basic-types.html)
- ¿QUÉ ES LA PROGRAMACIÓN FUNCIONAL? - BLOG DE CÓDIGO FACILITO. (N.D.). CÓDIGOFACILITO. [HTTPS://CODIGOFACILITO.COM/ARTICULOS/PROGRAMACION-FUNCIONAL](https://codigofacilito.com/articulos/programacion-funcional)
- CORVO, H. S. (2020, MARCH 19). PROGRAMACIÓN FUNCIONAL: CARACTERÍSTICAS, EJEMPLOS, VENTAJAS, DESVENTAJAS. LIFEDER. [HTTPS://WWW.LIFEDER.COM/PROGRAMACION-FUNCIONAL/](https://www.lifeder.com/programacion-funcional/)

THANK
YOU